

Desafio Técnico: Newsletter Inteligente

1. Qual é a Boa? O Desafio

Olá! Preparamos esse desafio técnico com o objetivo de conhecer melhor o seu raciocínio como desenvolvedor Pleno: como você pensa, escolhe e constrói soluções.

A proposta vai além de um front e back simples, queremos ver você pensando em toda a arquitetura da aplicação, incluindo comunicação assíncrona e integração com outras ferramentas.

O que mais nos interessa aqui são suas decisões técnicas:

Por que escolher determinado banco de dados SQL ou NoSQL? Qual message broker você utilizaria e com qual justificativa? Estamos muito interessados em entender seu raciocínio por trás dessas escolhas.

Além do domínio técnico, também vamos observar como você organiza seu código, utiliza o Git e documenta o projeto. Boas práticas de engenharia de software contam muito!

Linguagens preferenciais: Python e TypeScript.

Frameworks preferenciais: NestJS, FastAPI, React.

O Essencial vs. Os Extras

Olha, a gente sabe que parece bastante coisa! Então, para te dar um norte, o **essencial** é ter:

- Um **backend** que serve as notícias.
- Um **frontend** que mostra essas notícias.
- O **Agente** que faz a curadoria das notícias e salva no banco.

Tudo o que está descrito a mais (como o sistema de login completo, as preferências de usuário, a mensageria, o resumo com IA e os testes) é um **super bônus**! Se você entregar o essencial bem feito, já vai ser incrível. O resto é para quem quiser brilhar ainda mais.

2. O Que a Gente Espera de Você

Dá uma olhada no que esperamos ver em cada parte do projeto.

a. O Backend (A Cozinha do Projeto)

A API é o coração de tudo! A gente espera uma API REST bem organizada. O mínimo é ter a rota de notícias.

- **Mostrando as Notícias (Essencial):**

- GET /news: A rota principal para buscar as notícias.
- Tem que ter paginação, para não carregar tudo de uma vez.
- Dê a opção de filtrar por período: ?period=day|week|month.

- **Login e Cadastro (Bônus):**

- Uma rota para criar novos usuários (POST /users), pegando nome, e-mail e senha. Ah, e por favor, guarde a senha de forma segura (usando bcrypt, por exemplo!).
- Uma rota para fazer o login (POST /login) que devolve um token (tipo JWT) se tudo estiver certo.
- Proteja as rotas que só usuários logados podem acessar.

- **Preferências da Galera (Bônus):**

- GET /preferences: para listar todas as categorias de notícias que existem.
- GET /users/me/preferences: para mostrar as categorias que o usuário logado curte.
- PUT /users/me/preferences: para deixar o usuário mudar o que ele quer ver.

b. O Frontend (A Vitrine da Loja)

A interface tem que ser amigável e funcionar bem tanto no celular quanto no computador. Capricha no visual!

- **Página Principal (Essencial):**

- Mostre as notícias em cards, com título, de onde veio, um resuminho e a data.
- Coloque uns botões para o usuário filtrar por dia, semana ou mês.
- Pode usar o framework que você mais curte (React, Vue, Angular), o importante é ser uma SPA (Single Page Application).

- **Telas de Login e Preferências (Bônus):**

- Telas de Login/Cadastro e a de escolha de preferências quando o usuário entra pela primeira vez.

c. Onde Guardar os Dados (Banco de Dados)

SQL ou NoSQL? PostgreSQL ou MongoDB? A escolha é sua! O mais importante para gente é entender *por que* você escolheu uma tecnologia em vez da outra. Você vai precisar modelar como os dados serão guardados para as notícias, categorias e, se fizer os bônus, os usuários e suas preferências.

d. Comunicação Inteligente (Message Broker - Bônus)

Se quiser ir além, use um sistema de mensageria para mostrar que você manja de criar sistemas que aguentem o tranco.

- **Como funcionaria:**

1. O **Agente Curador** acha uma notícia e, em vez de salvar direto no banco, manda uma mensagem para uma fila (usando BullMQ, Redis ou similares).
2. Um serviço **Consumidor** fica de olho nessa fila, pega a notícia, processa (chama a IA etc.) e salva no banco. Isso deixa tudo mais organizado e desacoplado.

e. O Toque de Mágica (IA e o Agente)

- **O Agente Curador de Conteúdo (Essencial):**

- Aqui a gente quer ver você construindo um **agente high-code**. O que isso significa? Em vez de usar uma plataforma pronta de automação (low-code/no-code), você vai escrever o código que define a inteligência do agente.
- Ele deve ser um serviço separado (um worker) que segue regras complexas definidas por você para criar/selecionar notícias e salvá-las no banco. Por exemplo, pode ser um serviço que:
 - Gera notícias fictícias sobre tecnologia com base em templates.
 - Analisa um arquivo CSV ou JSON local para extrair insights e transformá-los em notícias.

- Processa textos para classificar sentimentos ou identificar entidades.
- A criatividade para definir a lógica do agente é um ponto chave da avaliação!
- **Resumos Automáticos com IA (Bônus):**
 - Se você implementar o sistema de mensageria, o "Consumidor" pode mandar o texto da notícia para uma IA (Claude, Gemini, GPT etc.) e pedir um resumo antes de salvar.

f. Rodando em Contêineres (Docker)

para gente ver que você também se vira com infra, a gente espera que a solução seja entregue de forma containerizada. Um docker-compose.yml que sobe todos os serviços (front, back, agente, banco de dados) seria o ideal!

g. Boas Práticas de Engenharia de Software

Além de tudo, vamos observar como você trabalha.

- **Testes Unitários (Bônus):** A gente adoraria ver testes para as partes críticas do seu backend. Isso mostra que você se preocupa com a qualidade e a manutenção do código.
- **Clareza no Código:** Um código limpo, bem-organizado e fácil de entender vale ouro. Comente o que for complexo, siga padrões de nomenclatura e estruture bem seus arquivos e pastas.
- **Uso do Git:** Queremos ver um histórico de commits bem-feito. Commits pequenos e com mensagens claras mostram como você organiza seu trabalho.
- **Documentação:** O README.md é a porta de entrada do seu projeto. Um README bem escrito, explicando as decisões e como rodar o projeto, faz toda a diferença.

3. Um Desenho para Ajudar

Para dar uma clareada em como as peças se encaixam, separamos a arquitetura em dois diagramas.

Diagrama Essencial

Este é o fluxo mínimo esperado para o desafio.

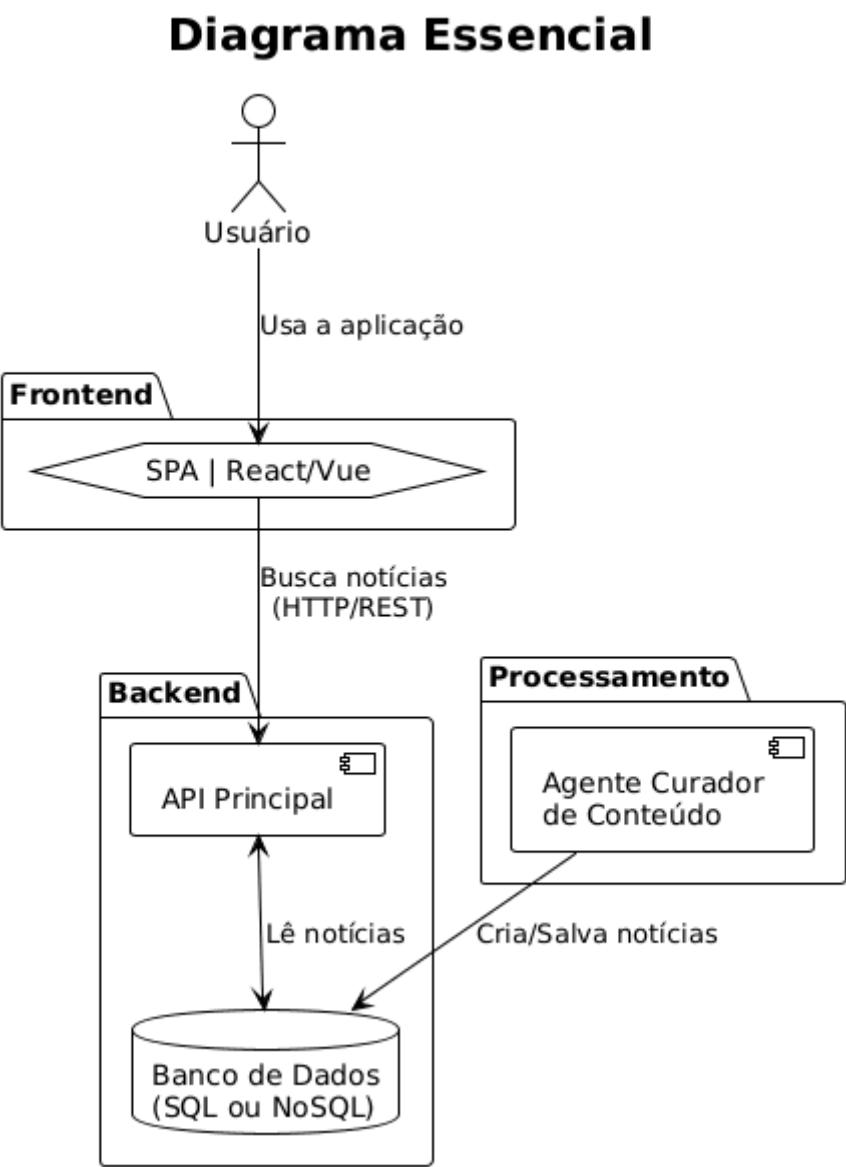
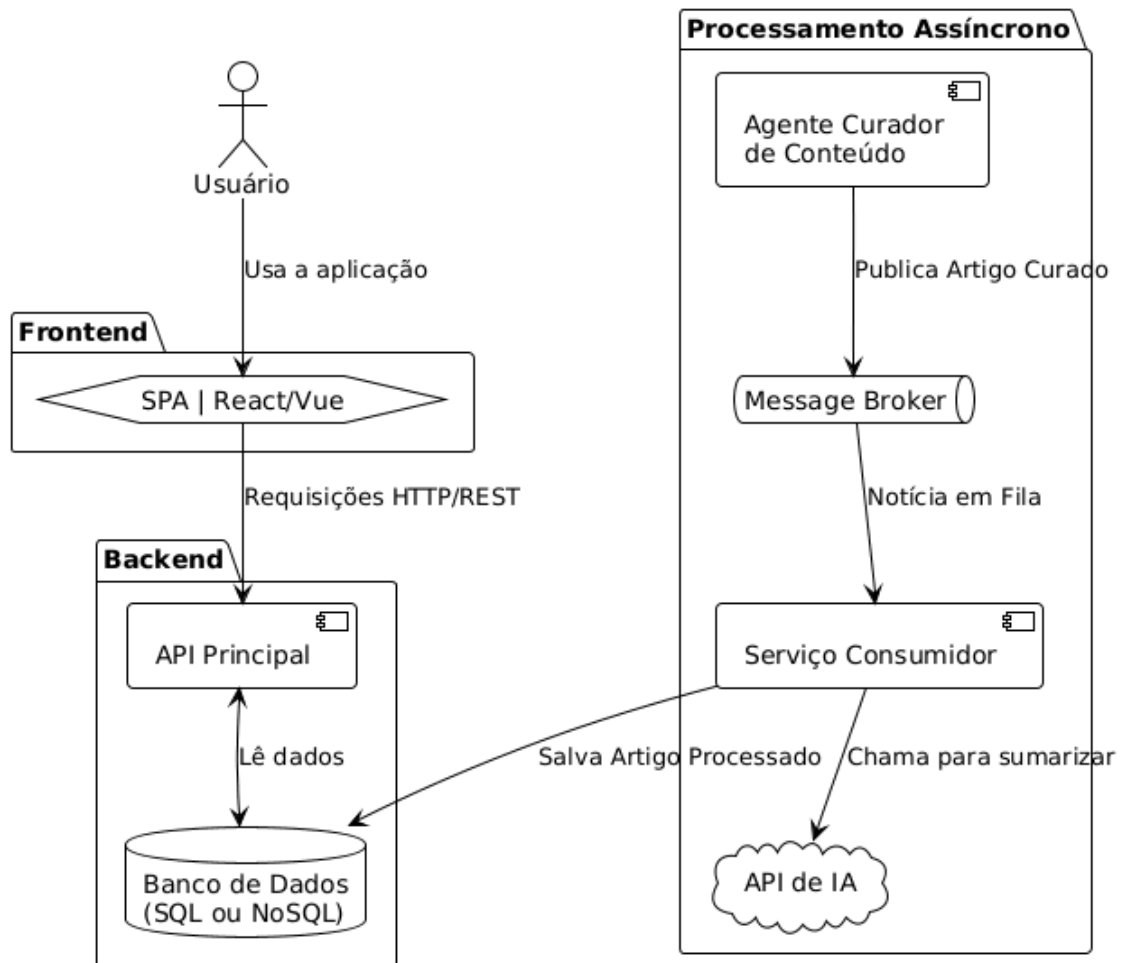


Diagrama Completo (com os Bônus)

Este é o fluxo se você decidir implementar todas as funcionalidades extras.

Diagrama Completo (com os Bônus)



4. Entrega

Você terá o prazo de 9 dias para entregar o teste, ao final você deverá enviar o link do seu repositório para o recrutador, que realizará a avaliação, se você concluir antes do prazo é só avisar quem está cuidando do seu recrutamento que ela passará a informação. Apoiamos o uso de inteligência artificial na cocriação do código. Se utilizar APIs não esqueça de descrever quais são e substituir as chaves por <CHAVE_DE_API>, para que seja fácil de encontrar e substituir.

BOA SORTE e esperamos que seja divertido desenvolver esse teste!